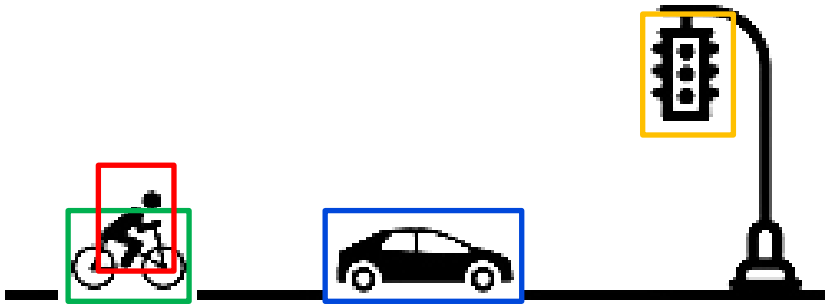
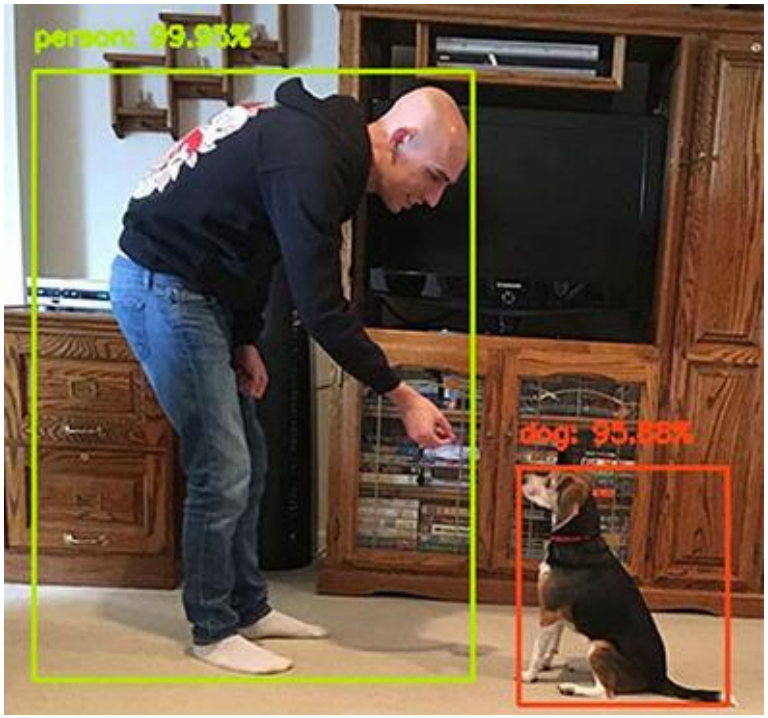
Deep Learning for Object Detection

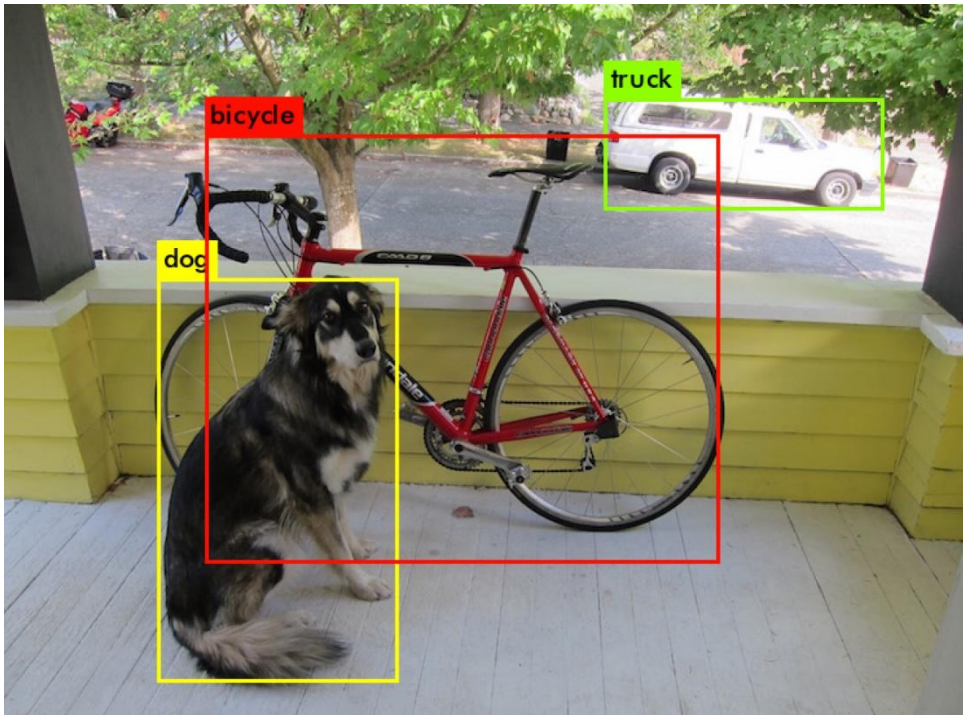


Matteo Ferrara
matteo.ferrara@unibo.it

Introduzione

Il processo di *object detection* restituisce una *bounding box*, l'etichetta di una classe (scelta tra un insieme predefinito) e spesso una probabilità (o *score*) per ogni oggetto di interesse presente nell'immagine di input.





Introduzione (2)

Dall'avvento di AlexNet diversi gruppi di ricerca hanno iniziato a studiare nuovi modelli per l'*object detection* basati su Convolutional Neural Network (CNN).

Valutazione delle prestazioni

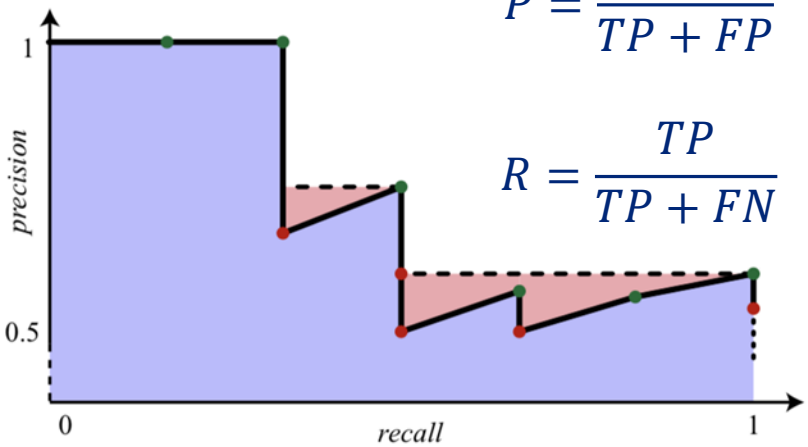
Possibili diversi tipi di errore, tra cui: oggetto non trovato, classe errata, bounding box sbagliata o imprecisa.

Come quantificare il tutto con uno scalare per rendere i sistemi facilmente confrontabili?

- *Average Precision* (AP): è calcolata come l'area sotto la curva *precision-recall* per oggetti di una singola classe su tutto il database. Solitamente la curva viene prima corretta rimpiazzando la precision al livello di recall r con la massima precision misurata con un valore di recall $\geq r$.

$$P = \frac{TP}{TP + FP}$$

$$R = \frac{TP}{TP + FN}$$



- *mean Average Precision* (mAP): è la media di AP su tutte le classi.

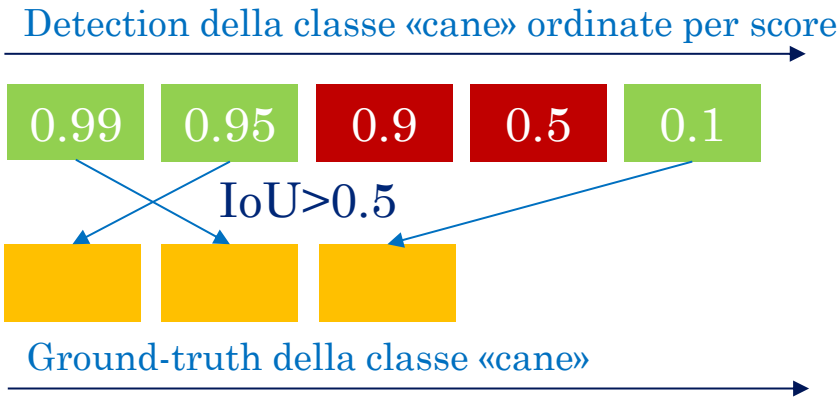
Valutazione delle prestazioni (2)

- Come calcolare TP, FP?
- Si considera una prediction corretta quando la classe è giusta e la *Intersection over Union* (IoU) delle due bounding box (rilevata e vera) è maggiore di un valore dato (solitamente 0.5 per ovviare a inesattezze del *ground truth*).

$$IoU = \frac{|At \cap Re|}{|At \cup Re|}$$

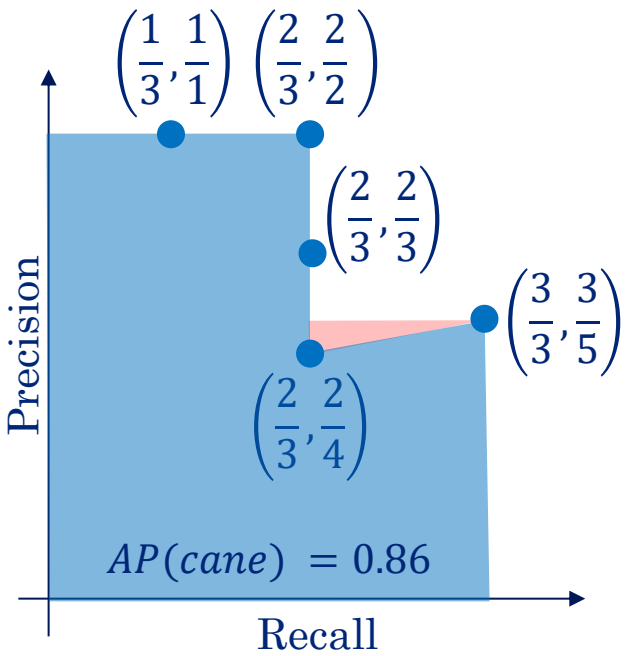
mean Average Precision: esempio

1. Eseguire la detection su tutte le immagini di test.
2. Calcolare la AP per ogni classe del problema.



$$Recall = \frac{3}{3} = 1.0$$

$$Precision = \frac{3}{5} = 0.6$$

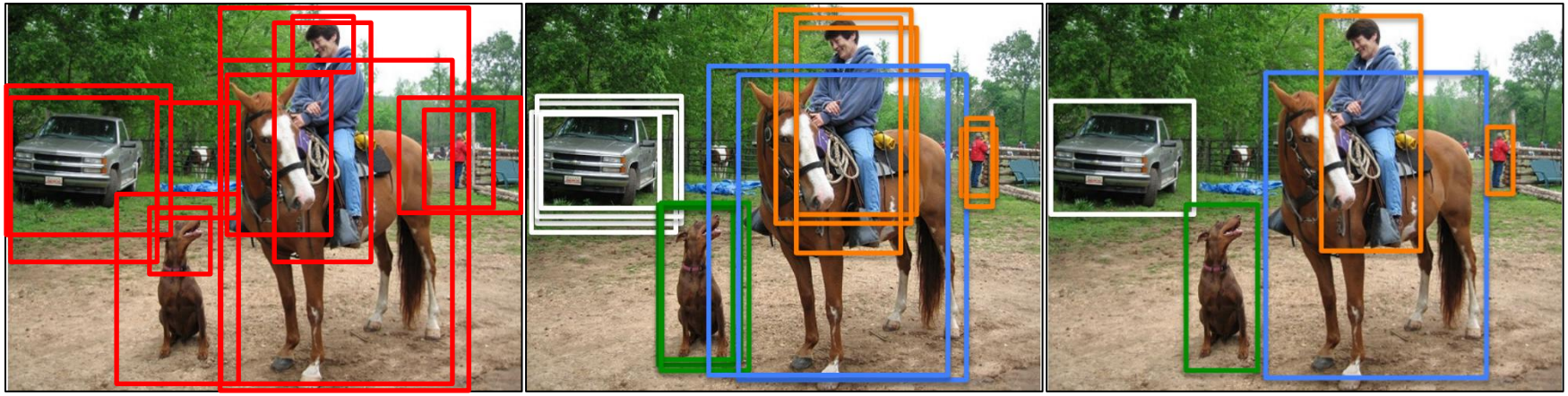


3. Calcolare la mAP come la AP media su tutte le classi del problema.

$$mAP = \frac{AP(cane) + AP(gatto) + AP(cavallo)}{3}$$

General framework

- Tipicamente un sistema di object detection presenta i seguenti componenti:
  1. Region proposal – identifica le regioni di interesse (RoI) che potrebbero contenere un oggetto.
  2. Object prediction – per ogni RoI determina se e quali oggetti sono presenti restituendo per ognuno bounding box e classe.
  3. Soppressione dei non-massimi – per ogni oggetto individuato analizza le bounding box associate mantenendo quella con la confidenza più alta e eliminando le altre.

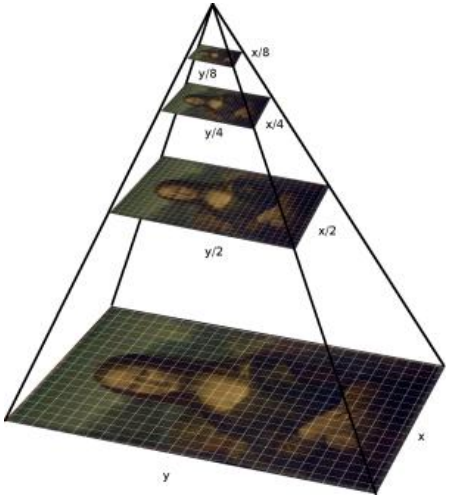


Sliding windows e image pyramids

Possiamo prendere una rete addestrata per la classificazione di immagini e utilizzarla per l'object detection?

1. Region proposal: sliding windows + image pyramids
2. Object prediction: pre-trained CNN

+



+

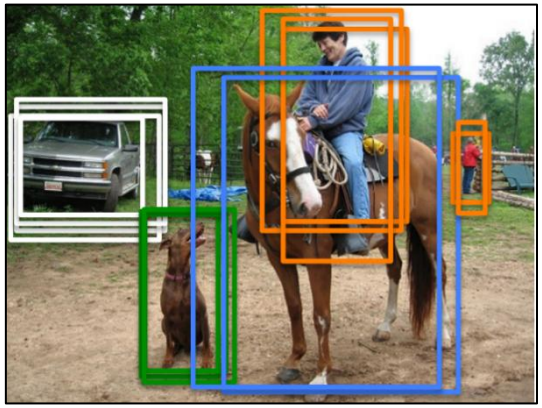
Pre-trained CNN

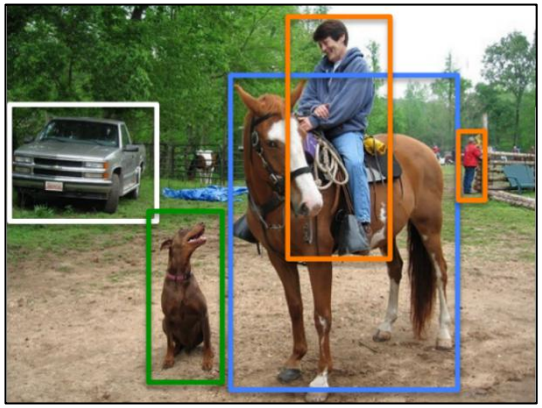
Sliding windows e image pyramids (2)

Dato un livello della piramide e una sliding window, la corrispondente sotto-immagine viene estratta e data in input alla rete che restituisce una classe L e una probabilità. Se la probabilità è superiore a una soglia T allora la sotto-immagine viene marcata come appartenente alla classe L.

Ripetendo questi passi per ogni livello della piramide e per ogni sliding window otteniamo un insieme di sotto-immagini candidate.

Infine tramite la soppressione dei non-massimi si eliminano le sotto-finestre con overlapping o con un basso grado di confidenza.





Sliding windows e image pyramids (3)

Contributi

- Intuitivo.
- Semplice da implementare.
- Possibilità di utilizzare diverse CNN pre-addestrate come *backbone* (es. AlexNet, VGGNet).

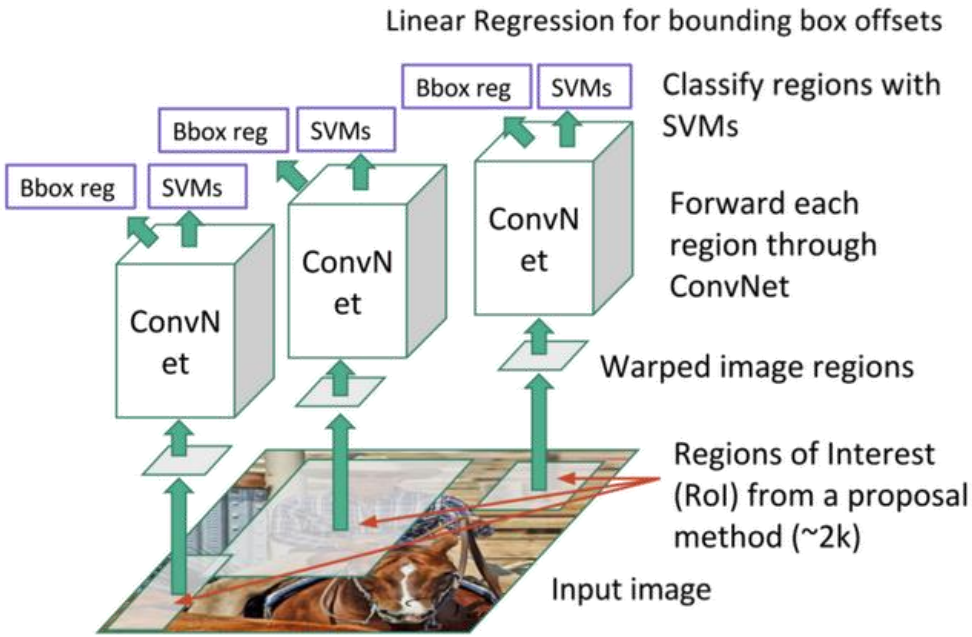
Svantaggi

- Il numero di finestre valutate è troppo elevato rendendo il sistema estremamente lento.
- Ricalcolo dell'intera CNN per ogni finestra.

R-CNN

Nel 2013 Girshick, Donahue, Darrell, e Malik hanno proposto R-CNN, un modello in due fasi composto da quattro moduli:

1. Individuazione delle regioni di interesse
   - region proposal – individua un insieme di regioni di interesse.
2. Classificazione di ogni regione
   a. estrattore di feature – estrae le feature da ogni regione utilizzando una CNN (*backbone*);
   b. classificatore – a partire dalle feature, per ogni regione restituisce l'etichetta della classe più probabile;
   c. regressore – ottimizza la bounding box restituita.



R. Girshick, J. Donahue, T. Darrell, and J. Malik, "Rich feature hierarchies for accurate object detection and semantic segmentation," *arXiv 1311.2524*, November 2013.

R-CNN (2)

- Per individuare le regioni di interesse viene utilizzato il metodo *selective search* (che approfondiremo più avanti) in grado di restituire circa 2000 regioni di interesse.
- Ogni regione viene ridimensionata a una dimensione fissa di 227 × 227 pixel (eventualmente modificandone l'aspect ratio).
- La regione ridimensionata viene data in input alla rete AlexNet (preaddestrata) ottenendo in output un vettore di 4096 feature (livello fully-connected *fc6* o *fc7*).
- Per ogni classe è stato addestrato un SVM lineare: le feature estratte da ogni regione sono valutate da tutti gli SVM ottenendo uno score per ogni classe.
- Per ogni classe è stato addestrato un regressore in grado di raffinare la posizione della bouding box.
- Tramite la soppressione dei non-massimi (effettuata separatamente per ogni classe) si rimuovono regioni con un elevato overlapping con altre regioni che presentano uno score più alto.

R-CNN (3)

Contributi

- Utilizzo di un algoritmo per la selezione preventiva delle regioni di interesse invece di eseguire una ricerca esaustiva su tutta l'immagine.

Svantaggi

- Modifica dell'aspect ratio di ogni regione può causare un calo dell'accuratezza in fase di estrazione delle feature.
- Ricalcolo dell'intera CNN per ogni regione.
- L'addestramento richiede molto tempo in quanto si dovrà effettuare il *fine-tuning* della CNN sulla base delle classi di interesse e successivamente dovranno essere addestrati gli SVM e i regressori.
- La fase di selezione delle regioni di interesse richiede circa 2 secondi a immagine e può restituire regioni spesso ridondanti.
- Troppo lenta per essere utilizzata in scenari real-time.

Selective search

L'algoritmo *selective search*, presa un'immagine in input, esegue i seguenti passi per produrre una gerarchia di regioni:

A. generazione di una segmentazione iniziale sulla base di colore e tessitura;

B. fusione di regioni simili
   1. selezione delle due regioni più simili tra loro (in termini di colore, tessitura, dimensione e posizione reciproca);
   2. fusione delle due regioni selezionate;
   3. si ripetono i passi 1 e 2 fino a quando non rimane un'unica regione.

Input image | Segmentazione iniziale | Dopo alcune iterazioni | Output

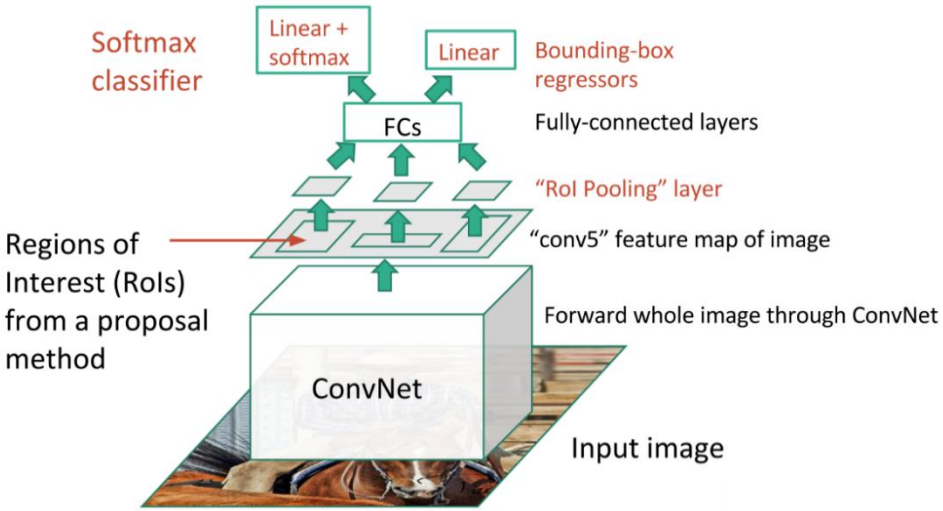
J. R. R. Uijlings, K. E. A. Sande, T. Gevers, and A. W. M. Smeulders, "Selective Search for Object Recognition," in *proceedings of the International Journal of Computer Vision*, vol. 104, no. 2, pp. 154-171, September 2013.

Fast R-CNN

Nel 2015 Girshick (Microsoft Research) ha proposto Fast R-CNN con l'intento di risolvere i problemi di R-CNN. Per fare ciò i tre componenti fondamentali di R-CNN (CNN, classificatore, bounding box regressor) sono stati inglobati tutti nella stessa architettura.

Fast R-CNN è composta da:

- una CNN pre-addestrata (es. VGG-16).
- l'ultimo livello di max pooling della CNN viene sostituito con un *Region of Interest* (RoI) pooling layer.
- l'ultimo livello fully-connected della CNN viene sostituito da due *sibling* layer di output.



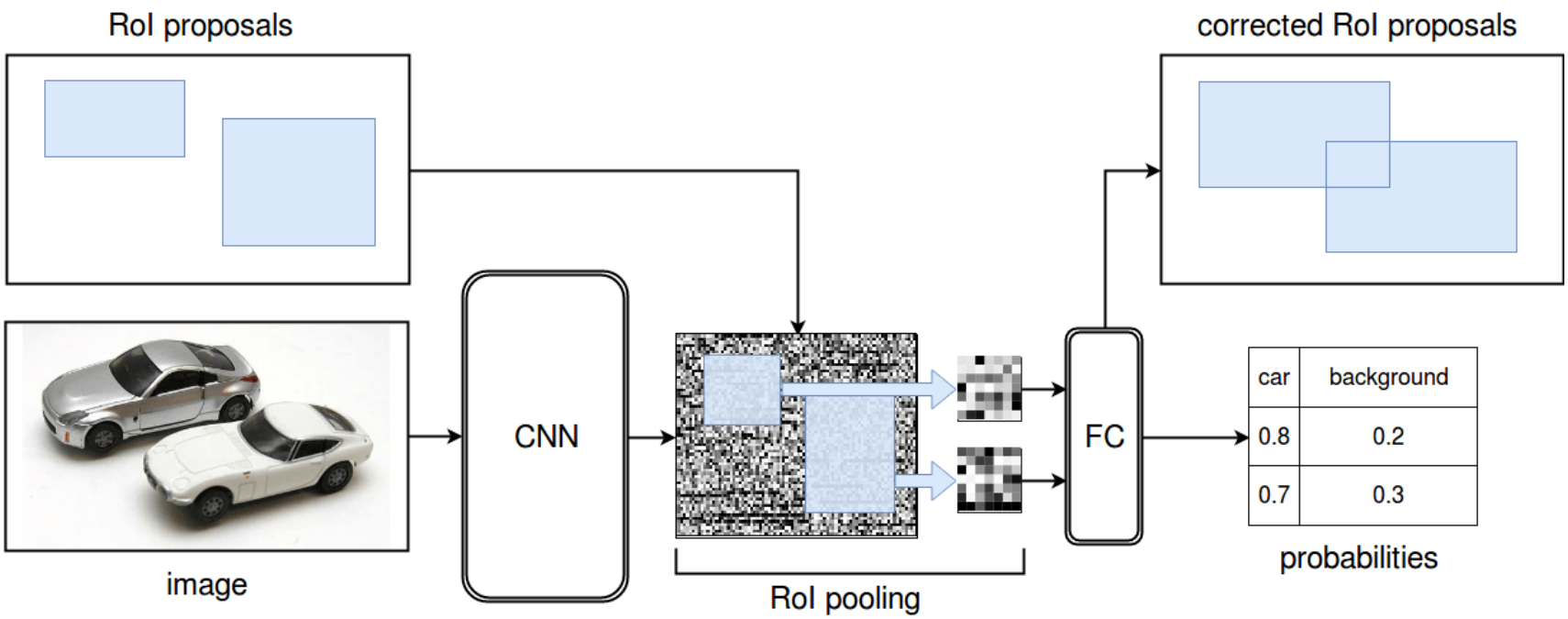
- La CNN viene modificata per prendere in input un'immagine e un insieme di regioni di interesse.

R. Girshick, "Fast R-CNN," *arXiv 1504.08083*, April 2015.

Fast R-CNN (2)

- Le regioni di interesse sono selezionate utilizzando l'algoritmo *selective search* utilizzato in R-CNN.
- L'immagine di input viene ridimensionata così che il lato corto abbia una dimensione di 600 pixel mentre il lato lungo non superi i 1000 pixel.
- L'immagine attraversa tutta la CNN fino a ottenere le feature map dell'ultimo convolutional layer.
- Il RoI pooling layer (che approfondiremo più avanti) riceve in input le feature map e le regioni di interesse e restituisce, per ogni regione di interesse, delle feature map di dimensione prefissata che sono passate in input ai successivi livelli fully-connected.
- Le feature ottenute dai livelli fully-connected vengono passate ai due sibling layer di output: il primo è il livello di classificazione che restituisce la classe di ogni regione di interesse mentre il secondo è il regressore che stima le coordinate delle bounding box di ogni oggetto individuato nell'immagine.
- Utilizza una *multi-task loss* (che approfondiremo più avanti) per addestrare contemporaneamente la rete a predire la classe e la corrispondente bounding box degli oggetti presenti nell'immagine.

Fast R-CNN (3)


RoI proposals
corrected RoI proposals
image
CNN
RoI pooling
FC
car
background
0.8
0.2
0.7
0.3
probabilities

Fast R-CNN (4)

Contributi

- Addestramento end-to-end.
- Introduzione del Region of Interest pooling layer.
- Migliori prestazioni (accuratezza/efficienza) rispetto a R-CNN.
- Addestramento con multi-task loss.

Svantaggi

- Utilizzo di un metodo esterno alla rete per la selezione delle regioni di interesse (che richiede circa 2s ad immagine).
- Troppo lenta per essere utilizzata in scenari real-time.

Region of Interest pooling layer

Uno dei principali ostacoli per utilizzare CNN (progettate per image classification) per problemi di object detection, è la necessità di utilizzare input di dimensioni prefissate (a causa dei livelli fully-connected presenti in queste reti) quando il numero di oggetti e le loro dimensioni è variabile e non conosciuto a priori.

Per ovviare a questo problema, il RoI pooling layer è in grado di restituire più volumi di output di dimensione prefissata partendo da:

- una serie di feature map (volume di input);
- N regioni di interesse.

Per ogni regione di interesse, la porzione corrispondente delle feature map di input viene convertita in un insieme di feature map a dimensione prefissata tramite max pooling.

Il numero di canali di output corrisponde al numero di canali di input mentre le dimensioni spaziali dell'output non dipendono né dalle dimensioni di input né dalle dimensioni delle regioni di interesse ma solamente dagli iperparametri del layer: dimensioni del volume di output (P_W e P_H) e fattore di scala (P_S).

Region of Interest pooling layer (2)

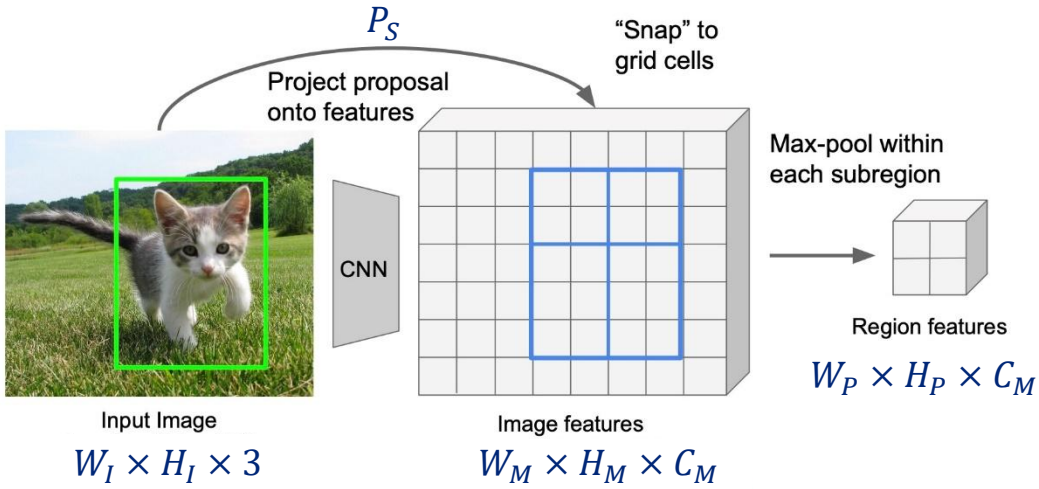
Siano W_I e H_I le dimensioni spaziali di input della rete mentre $W_M \times H_M \times C_M$ le dimensioni del volume di input del RoI pooling layer.

Le regioni di interesse sono state individuate sull'immagine di input della rete e pertanto dovranno essere riscalate sulle dimensioni spaziali del volume di input del RoI pooling layer (fattore di scala $P_S = \frac{W_M}{W_I} = \frac{H_M}{H_I}$).

Ogni regione di interesse viene riscalata e la corrispondente porzione di feature map suddivisa in $W_P \times H_P$ blocchi.

In ogni blocco viene selezionato il massimo di ogni canale e copiato nel volume di output.

Per tutte le N regioni di interesse l'output avrà sempre dimensione $W_P \times H_P \times C_M$.



Region of Interest pooling layer (3)

Esempio:

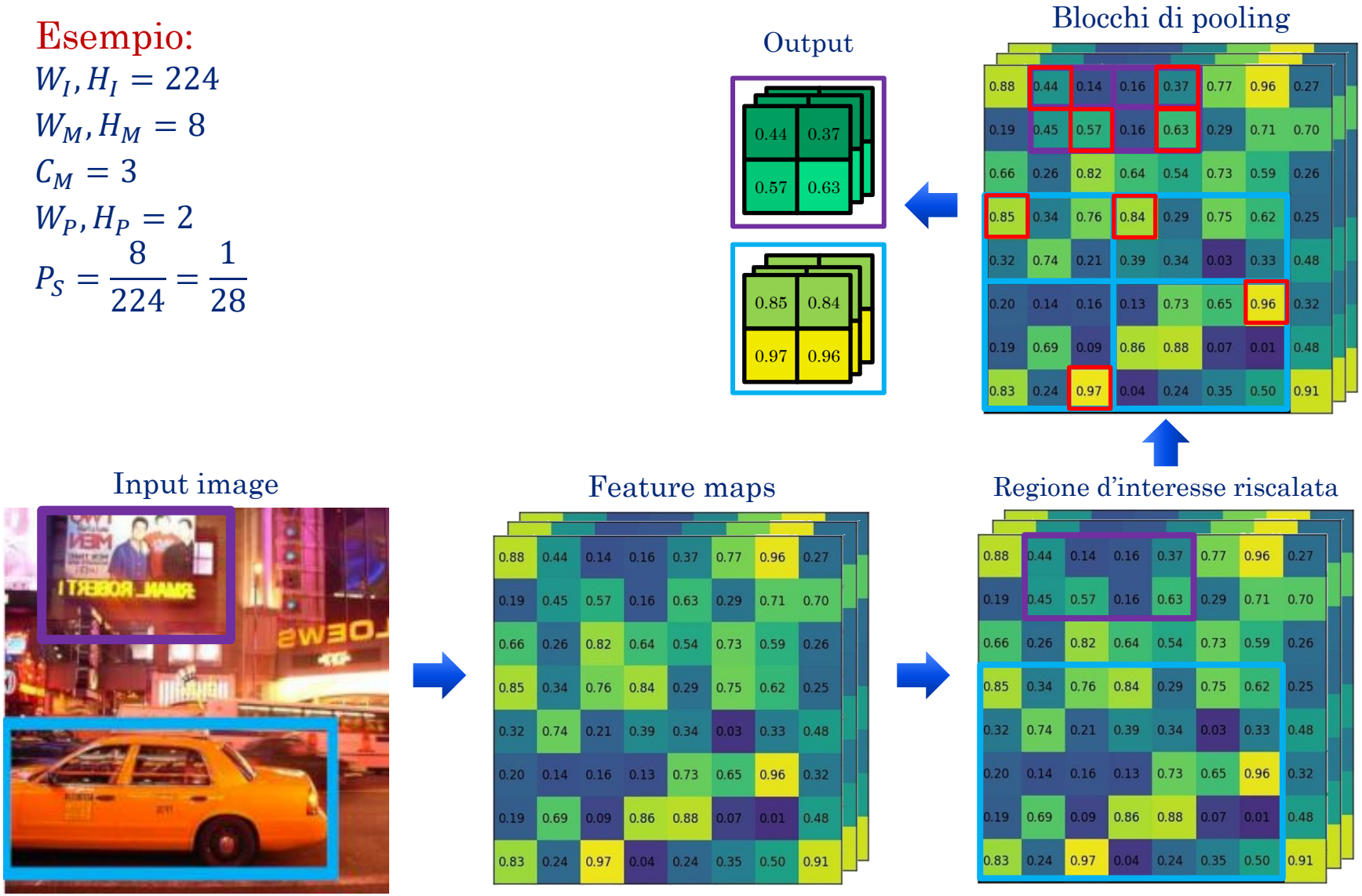
$W_I, H_I = 224$

$W_M, H_M = 8$

$C_M = 3$

$W_P, H_P = 2$

$$P_S = \frac{8}{224} = \frac{1}{28}$$



Multi-task loss

Fast R-CNN è un'architettura end-to-end in grado di apprendere contemporaneamente sia la classe che la bounding box degli oggetti di interesse presenti nell'immagine. Per fare ciò, viene utilizzata una multi-task loss.

Il layer di classificazione restituisce un valore di probabilità $p = p_0, \dots, p_K$ (con p_0 probabilità della classe background) per ogni classe. Dato il vettore delle probabilità p e la classe vera u la loss del livello di classificazione è:

$$L_{cls}(p, u) = -\log(p_u).$$

Data una classe u (con $u \geq 1$, la classe background non ha bounding box), il livello di regressione produce una tupla $t^u = (t_x, t_y, t_w, t_h)$ i cui elementi rappresentano le coordinate del corner superiore sinistro, la larghezza e l'altezza della bounding box. Indicando con $v = (v_x, v_y, v_w, v_h)$ la bounding box vera della classe u, come loss del livello di regressione viene utilizzata una L1 loss (smoothed):

$$L_{loc}(t^u, v) = \sum_{i \in x,y,w,h} smooth_{L_1}(t_i^u - v_i)$$

$$smooth_{L_1}(x) = \begin{cases} 0.5 \cdot x^2 & \text{if } |x| < 1 \\ |x| - 0.5 & \text{otherwise} \end{cases}$$

Multi-task loss (2)

La multi-task loss è ottenuta combinando insieme le due loss:

$$L(p, u, t^u, v) = L_{cls}(p, u) + \lambda[u \geq 1] \cdot L_{loc}(t^u, v)$$

$$[u \geq 1] = \begin{cases} 1 & \text{if } u \geq 1 \\ 0 & \text{otherwise} \end{cases}$$

La funzione $[u \geq 1]$ serve per annullare il contributo della loss del livello di regressione se la classe vera u rappresenta il background.

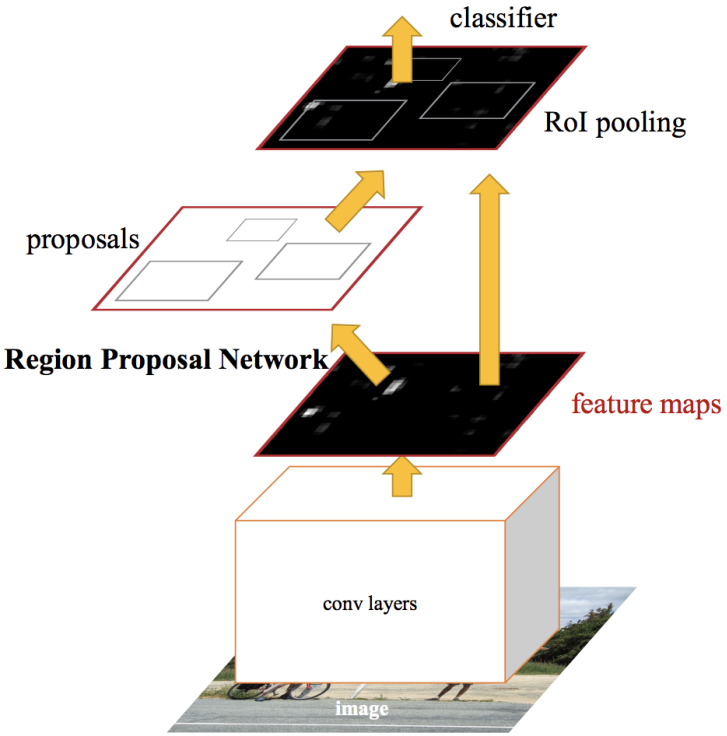
L'iperparamentro λ controlla il bilanciamento tra le due loss L_{cls} e L_{loc}.

Faster R-CNN

Sempre nel 2015, Ren, He, Girshick e Sun hanno proposto Faster R-CNN che, a differenza di R-CNN e Fast R-CNN, non utilizza nessun metodo esterno per individuare le regioni di interesse.

Faster R-CNN è composta da:

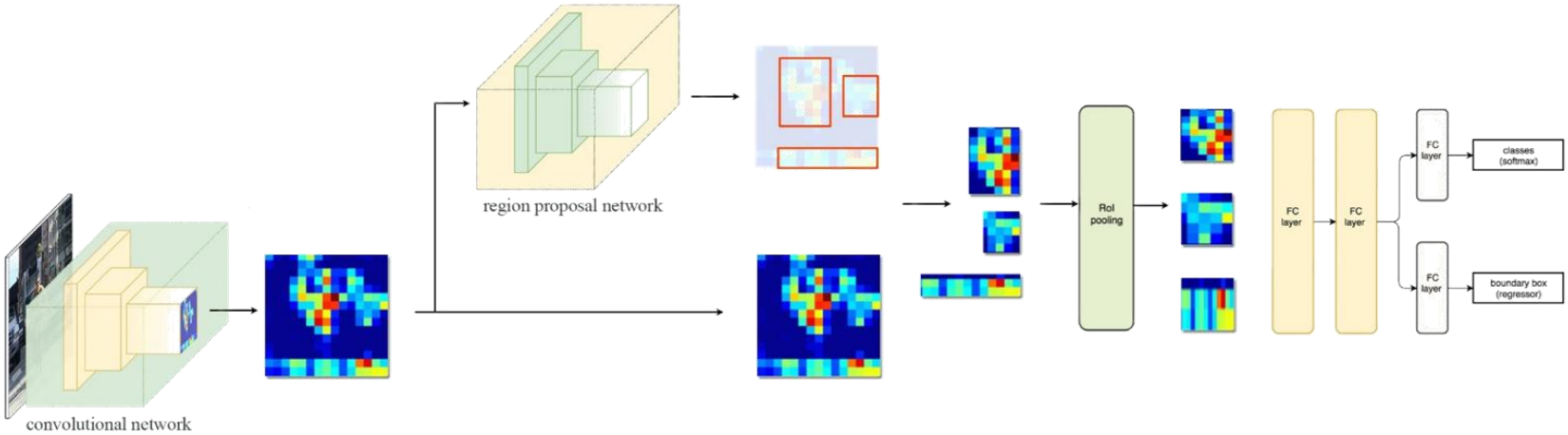
- una Fast R-CNN.
- una *Region Proposal Network* (RPN).



S. Ren, K. He, R. Girshick, J. Sun, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," *arXiv 1506.01497*, June 2015.

Faster R-CNN (2)

- L'immagine di input viene ridimensionata così che il lato corto abbia una dimensione di 600 pixel mentre il lato lungo non superi i 1000 pixel.
- L'immagine attraversa tutta la CNN (della Fast R-CNN) fino a ottenere le feature map dell'ultimo convolutional layer.
- La Region Proposal Network (che approfondiremo più avanti) riceve in input le feature map e restituisce un insieme di regioni di interesse.
- L'esecuzione prosegue come una normale Fast R-CNN che utilizza le regioni di interesse restituite dalla RPN (invece che dal metodo esterno *selective search).*



Faster R-CNN (3)

Contributi

- Introduzione della Region Proposal Network.
- Migliori prestazioni (accuratezza/efficienza) rispetto a Fast R-CNN.

Svantaggi

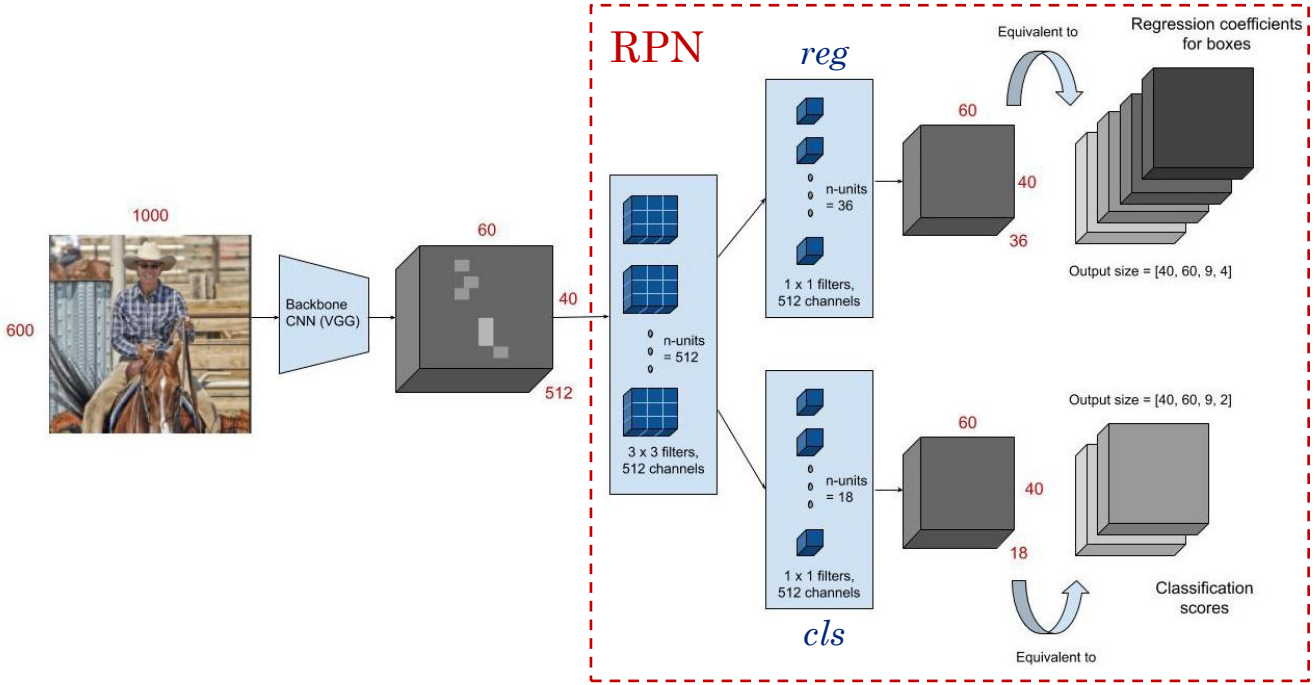
- Troppo lenta per essere utilizzata in scenari real-time.

Region Proposal Network

Dato in input il volume di dimensione $W_F \times H_F \times C_F$, ottenuto dall'ultimo convolutional layer della CNN a partire da un'immagine di input di dimensioni $W_I \times H_I$, la *Region Proposal Network* (RPN) restituisce un insieme di regioni di interesse.

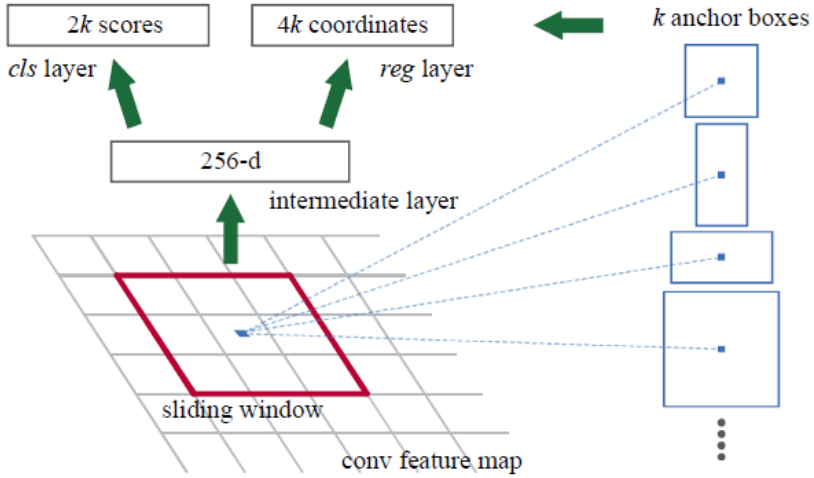
La rete è composta da:

- un convolutional layer con d kernel 3×3.
- due *sibling* convolutional layer (*reg* e *cls*) con $4k$ e $2k$ kernel 1×1.



Region Proposal Network (2)

- Il primo convolutional layer simula una sliding window 3×3 da cui vengono estratte d feature per ogni possibile posizione (pari a un volume di output di dimensione $W_F \times H_F \times d$).
- Ad ogni posizione della sliding window si vuole che la RPN valuti k possibili proposte di regione (o ancore).
- Il primo sibling convolutional layer (*reg*) restituisce un volume di output di dimensione $W_F \times H_F \times 4k$ che rappresenta, per ogni posizione della sliding window, i 4 coefficienti di regressione (coordinate del centro, larghezza e altezza) per ognuna delle k regioni.
- Il secondo sibling convolutional layer (*cls*) restituisce un volume di output di dimensione $W_F \times H_F \times 2k$ che rappresenta la probabilità che vi sia o meno un oggetto di interesse in ognuna delle k regioni della sliding window.



Region Proposal Network (3)

- Il parametro d è stato impostato a 256 utilizzando AlexNet come *backbone* o 512 con VGG-16.
- Sono state usate 9 (k) ancore per ogni posizione corrispondenti a 3 diverse scale (128×128, 256×256 e 512×512) e 3 diversi aspect ratio (1:1, 2:1, 1:2).
- Considerando immagini di input di dimensioni 1000×600 le feature map di input della RPN avranno dimensione 60×40 per un totale di $60 \times 40 \times 9 = 21600$ ancore.
- Durante la fase di training:
  - tutte le ancore che si intersecano con il bordo dell'immagine sono scartate riducendo così il loro numero a circa 6000 per immagine.
  - un'ancora è etichettata come contenente un oggetto se i) presenta il più alto valore di IoU con una regione del ground truth oppure ii) presenta un valore di IoU≥0.7 con una qualsiasi regione del ground truth.
  - un'ancora è etichettata come di background se presenta un valore di IoU<0.3 con tutte le regioni del ground truth.
  - tutte le altre ancore non contribuiscono all'addestramento.

Region Proposal Network (4)

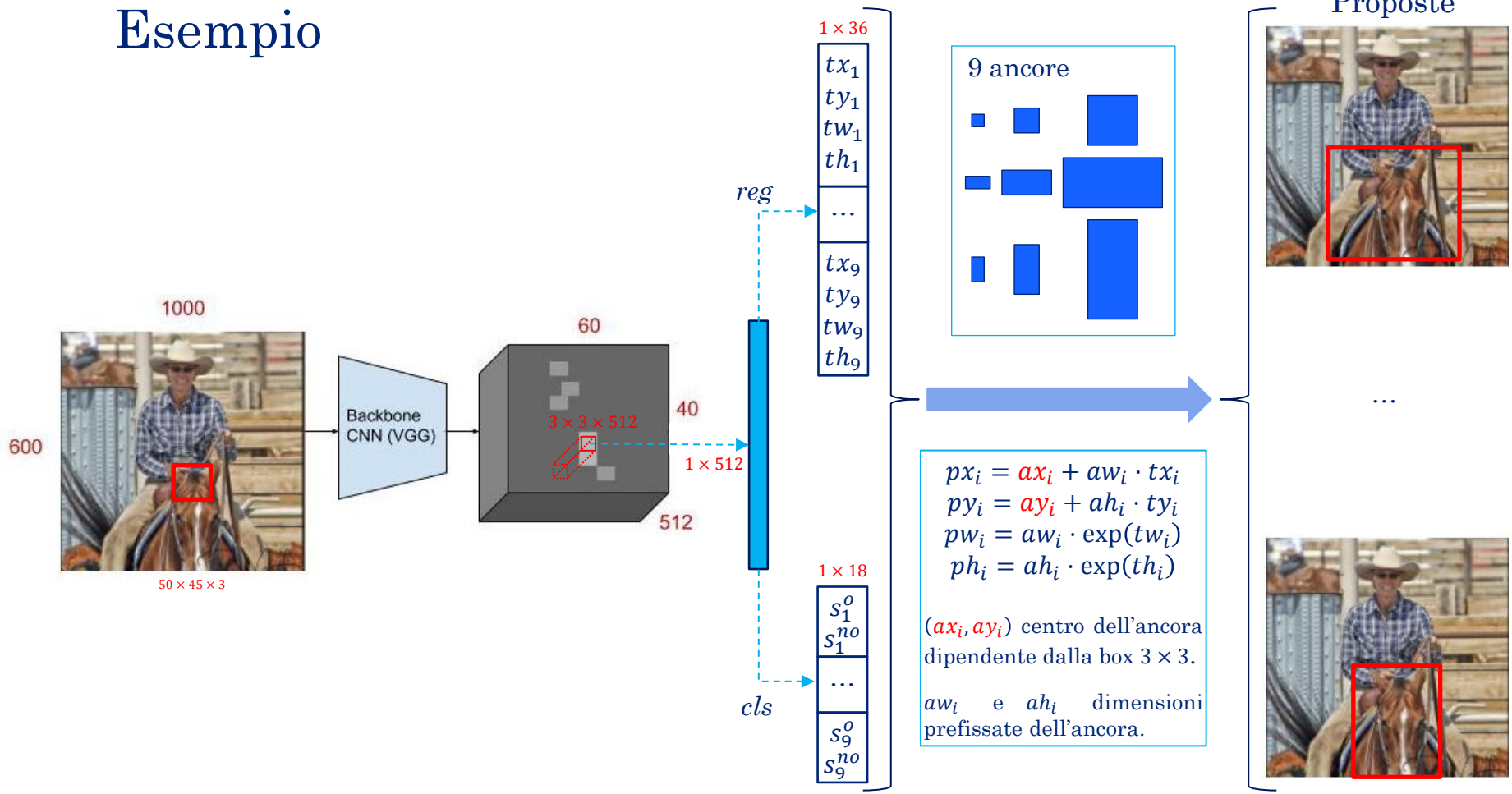
- Data un'immagine, l'output dei layer *cls* e *reg* sono due vettori $\{p_i\}$ e $\{t_i\}$ in cui, per ogni ancora i, p_i è la probabilità che contenga un oggetto mentre la tupla t_i contiene le coordinate della corrispondente regione di interesse.
- La multi-task loss è definita come:

$$L(\{p_i\},\{t_i\}) = \frac{1}{N_{cls}} \cdot \sum_i L_{cls}(p_i, p_i^*) + \frac{\lambda}{N_{reg}} \cdot \sum_i \left(p_i^* \cdot L_{reg}(t_i, t_i^*)\right)$$

- p_i^* e t_i^* rappresentano il ground truth dell'ancora i: p_i^* uguale a 1 se l'ancora contiene un oggetto o 0 altrimenti mentre t_i^* contiene i parametri della regione di interesse corrispondente (se $p_i^* = 1$).
- L_{cls} e L_{reg} sono le stesse loss utilizzate nella Fast R-CNN.
- I due termini sono normalizzati tramite N_{cls}(dimensione dei mini-batch - 256) e N_{reg} (numero di ancore ~ 2400) e pesati utilizzando λ (10).
- Durante la fase di detection, le regioni restituite dalla RPN possono presentare un'elevata sovrapposizione. Per ovviare a questo problema è stata adottata una tecnica di soppressione dei non-massimi basata sullo score di classificazione.

Region Proposal Network (5)

Esempio



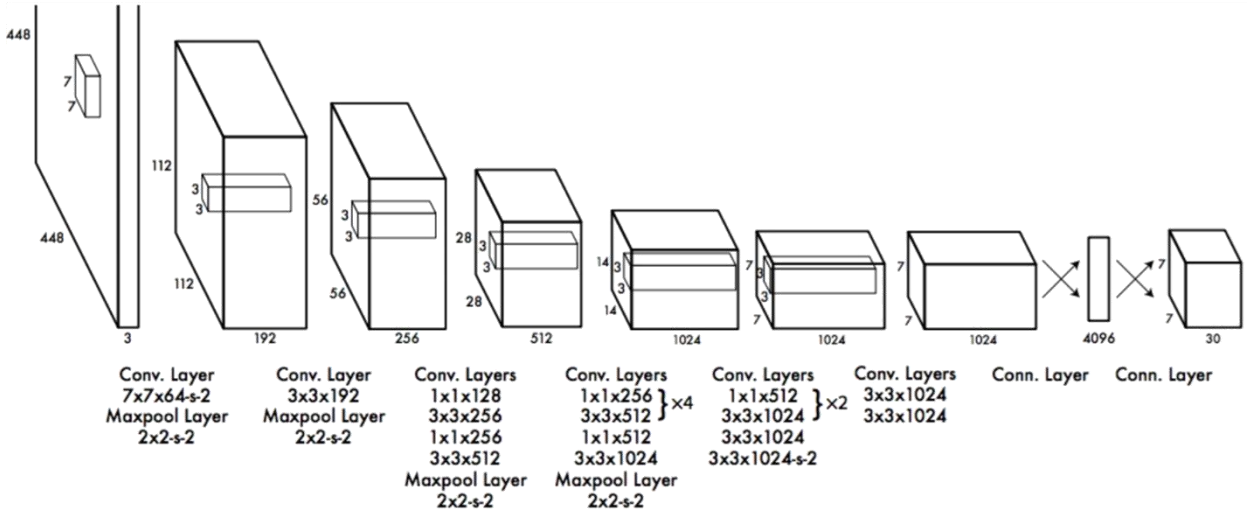
Multi-stage vs. single-stage

- Tutti i modelli visti fino a ora sono *multi-stage*:
  1. vengono identificate un insieme di regioni di interesse;
  2. solo le regioni candidate vengono processate.
- I modelli *single-stage* effettuano la detection su una serie di posizioni predefinite evitando di individuare preventivamente le regioni di interesse:
  - YOLO
  - SSD
  - RetinaNet
- Generalmente i modelli single-stage sono meno accurati ma più veloci dei modelli multi-stage.

YOLO

Nel 2015, Redmon, Divvala, Girshick e Farhadi hanno proposto il modello YOLO (acronimo di *You Only Look Once*), una rete single-stage in grado di lavorare in scenari real-time evitando di ricercare possibili regioni di interesse. È composta da:

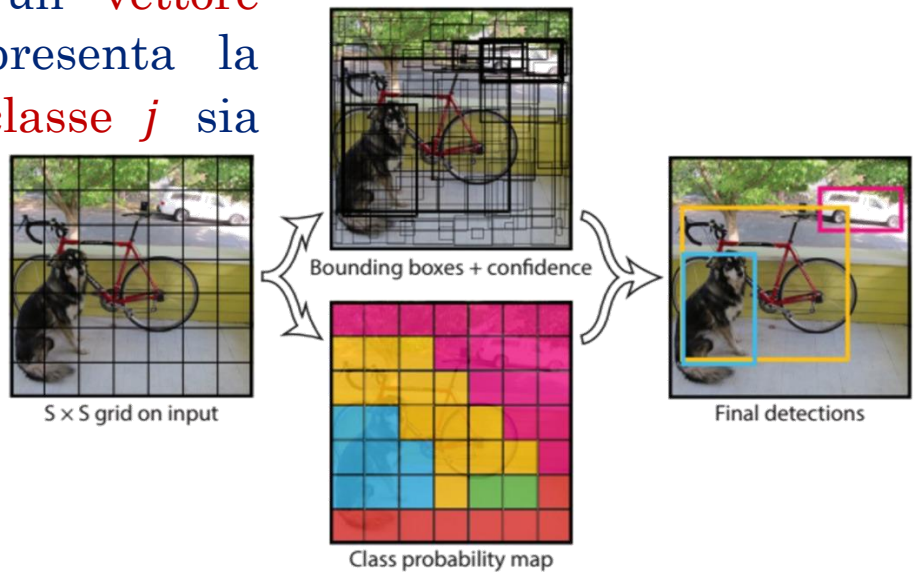
- 24 convolutional layer;
- quattro livelli di max pooling;
- due livelli fully-connected.



J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," *arXiv 1506.02640*, June 2015.

YOLO (2)

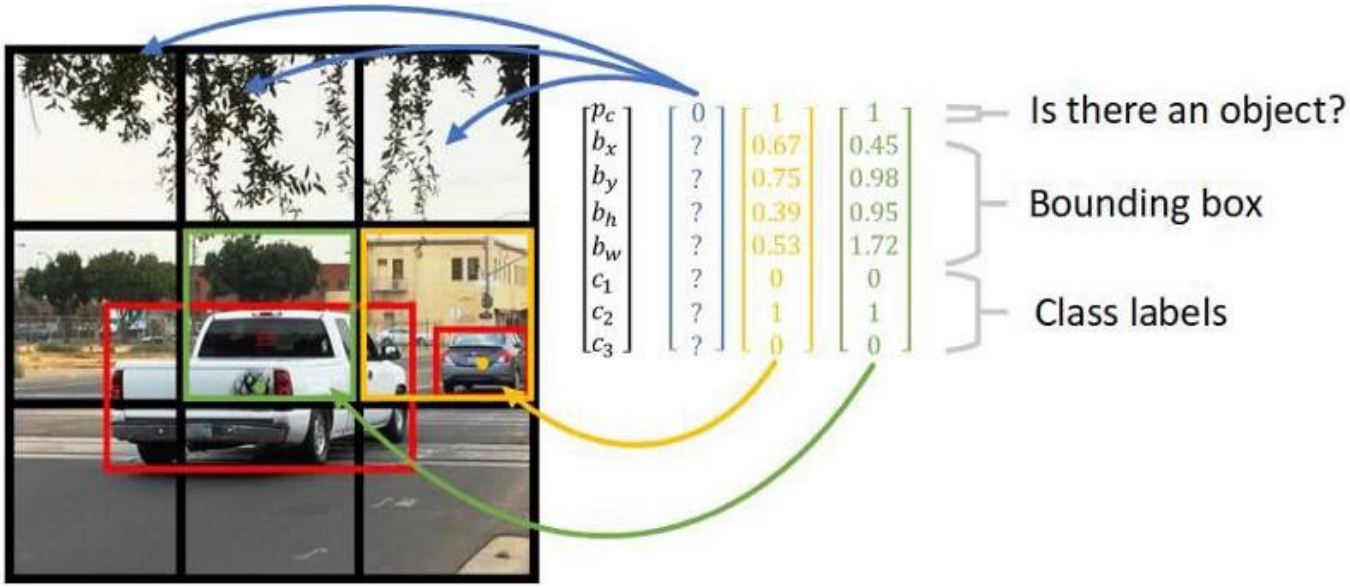
- L'immagine di input viene riscalata a una dimensione di 448×448 e suddivisa con una griglia di $S \times S$ celle ($S = 7$).
- La cella responsabile dell'individuazione di un oggetto è quella in cui ricade il centro di tale oggetto.
- Ogni cella restituisce per B bounding box ($B = 2$) una tupla $[c, x, y, w, h]$ contenente un valore di confidenza (c) che indica quanto il modello sia certo che la bounding box contenga un oggetto e le coordinate del centro della bounding box (x, y), le sue dimensioni (w, h).
- Inoltre, ogni cella restituisce un vettore $[p_j], 1 \leq j \leq C$ in cui p_j rappresenta la probabilità che un oggetto di classe j sia presente all'interno della cella.



YOLO (3)

- La probabilità che la bounding box i contenga un oggetto di classe j è calcolata come: $c_i \cdot p_j$.
- Tali predizioni sono restituite sotto forma di un volume di output di dimensioni $S \times S \times (5 \cdot B + C)$.
- Infine, viene applicata una tecnica di soppressione dei non-massimi per ridurre il numero di regioni di interesse individuate utilizzando una soglia sulla confidenza del modello.

Esempio:
$S = 3$
$B = 1$
$C = 3$



YOLO (4)

Contributi

- In grado di processare immagini in real-time.

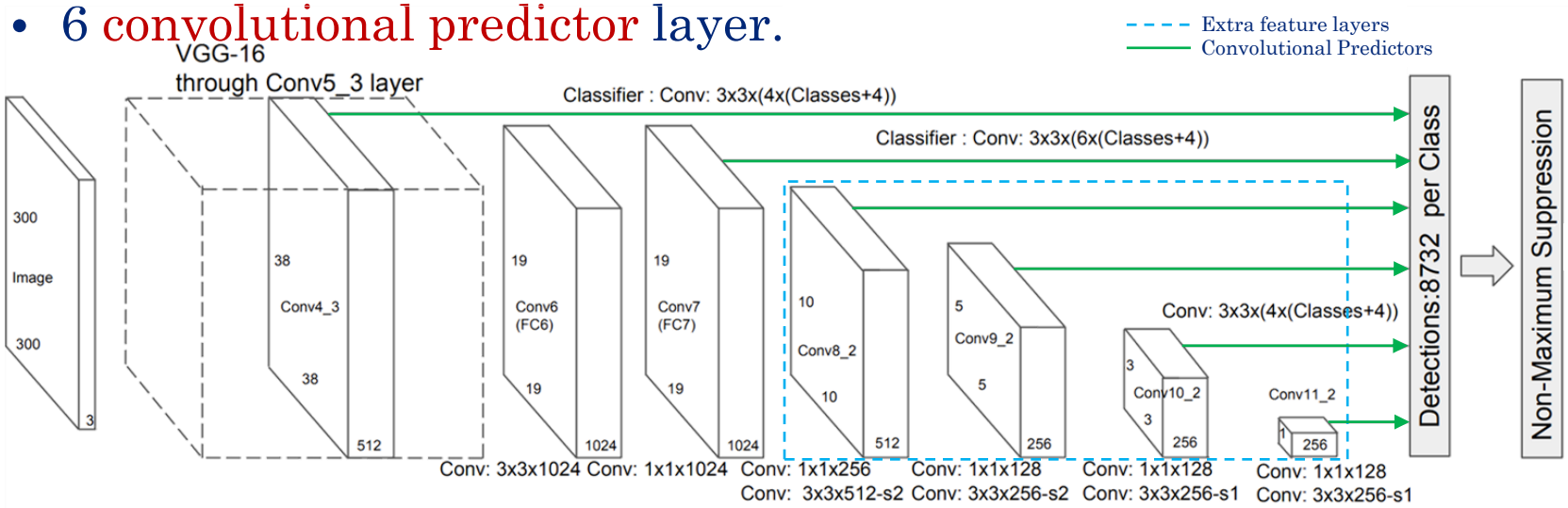
Svantaggi

- Meno accurata di Faster R-CNN.
- Non è in grado di individuare oggetti multipli nella stessa cella.
- Lo stesso oggetto potrebbe essere individuato più volte in celle adiacenti.

SSD

Nel 2015, è stato proposto il modello single-stage SSD (acronimo di Single-Shot multibox Detector) costituito da tre componenti principali:

- VGG-16 come backbone;
- quattro extra feature layer (ognuno composto da due convolutional layer);
- 6 convolutional predictor layer.



W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C. Y. Fu, and A. C. Berg, "SSD: Single Shot MultiBox Detector," *arXiv 1512.02325*, December 2015.

SSD (2)

- L'immagine di input viene riscalata a una dimensione di 300 × 300.
- Utilizza i livelli superficiali (con un receptive field piccolo) per rilevare piccoli oggetti e i livelli profondi (che coprono un receptive field maggiore) per identificare oggetti grandi.
- Una serie di 8732 bounding box con dimensioni, aspect ratio e posizioni differenti (*default box* - che approfondiremo più avanti) vengono valutate dalla rete restituendo, per ognuna di queste, quattro offset e un valore di confidenza per ogni classe.
- Sia gli offset che le confidenze vengono calcolate tramite *convolutional predictor* (che approfondiremo più avanti) applicati a diverse feature map.
- Infine, viene applicata una tecnica di soppressione dei non-massimi per restituire al massimo 200 regioni di interesse. Vengono eliminate quelle regioni che presentano una IoU maggiore di 0.45 con altre regioni della stessa classe.

SSD (3)

Contributi

- Accuratezza confrontabile con Faster R-CNN.
- In grado di processare immagini in real-time.
- Più veloce e più accurata di YOLO.
- Introduzione delle default box e della localizzazione multi-scala.

Svantaggi

- Non funziona bene su oggetti piccoli perché i livelli superficiali della rete non sono in grado di generare feature di alto livello.
- Richiede una grande quantità di dati per essere addestrata.

Default box

- Le default box rappresentano delle bounding box predefinite sulla base delle loro dimensioni, dell'aspect ratio e della posizione.
- L'obiettivo del modello è di decidere quali default box utilizzare per una data immagine e predire gli offset da applicare per ottenere la regione di interesse finale.
- Feature map ottenute da diversi livelli della rete hanno dimensioni e receptive field differenti pertanto le default box dovranno essere ridimensionate opportunamente. Supponiamo di voler utilizzare feature map da m livelli diversi, il fattore di scala ($s_k, k \in [1, m]$) per ogni livello è definito come:

$$s_k = s_{min} + \frac{s_{max} - s_{min}}{m - 1} \cdot (k - 1)$$

- Utilizzando $s_{min} = 0.2$, $s_{max} = 0.9$ e i seguenti aspect ratio $a_r \in \left\{1,2,3,\frac{1}{2},\frac{1}{3}\right\}$ le dimensioni delle default box sono così calcolate:

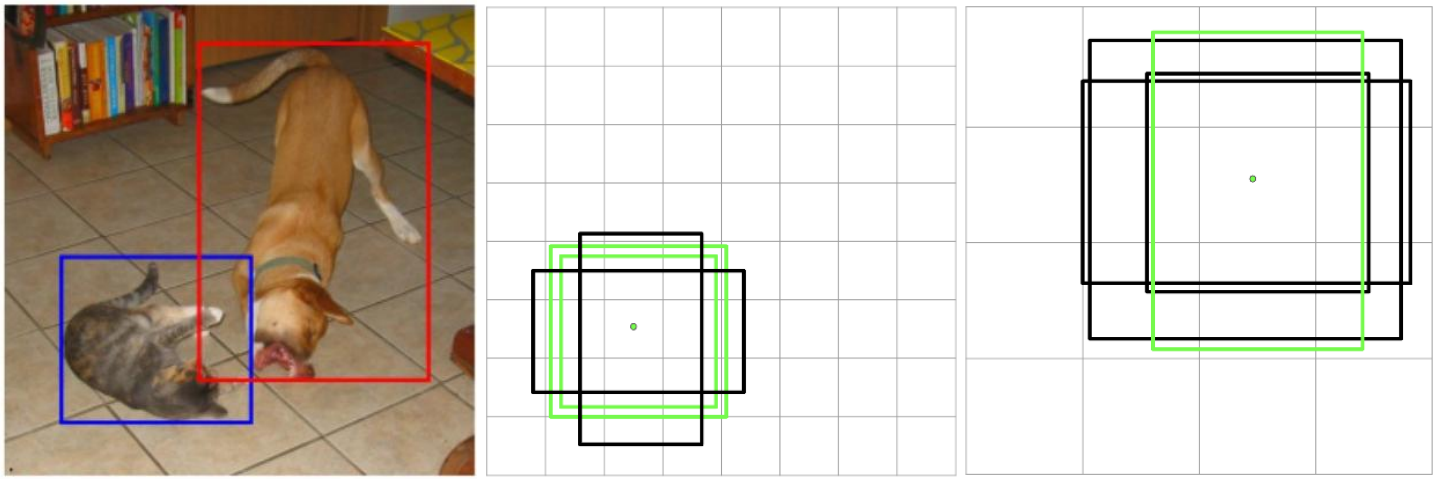
$$w_k^r = s_k \cdot \sqrt{a_r} \text{ e } h_k^r = \frac{s_k}{\sqrt{a_r}}$$

Default box (2)

Esempio:

- L'immagine sottostante contiene due oggetti con scala e aspect ratio differenti e rispettive regioni del ground truth.
- Sono riportate due feature map con dimensioni e receptive field differenti su cui sono visualizzate alcune default box.
- Le bounding box verdi (*positive box*) sono default box con un valore di IoU maggiore di 0.5 con almeno una regione del ground truth.

Input image | 8 × 8 feature map | 4 × 4 feature map

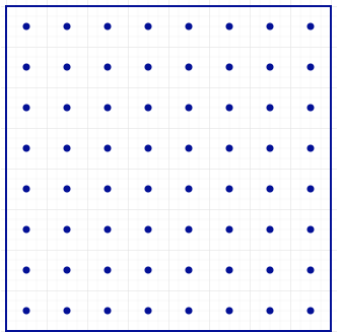


Convolutional predictor

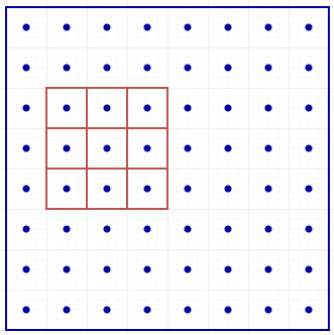
- Dato un volume di input di dimensione $W_F \times H_F \times C_F$, k default box e c possibili classi, applicando $k \cdot (4 + c)$ kernel $3 \times 3 \times C_F$ si ottiene un volume di output di dimensione $W_F \times H_F \times \big(k \cdot (4 + c)\big)$.
- In ogni posizione spaziale del volume di output, per ognuna delle k default box vi sono 4 offset rispetto alla corrispondente default box e c confidenze (una per ogni classe).

Esempio:
$W_F = H_F = 8$
$C_F = 1$
$k = 3$
$c = 2$

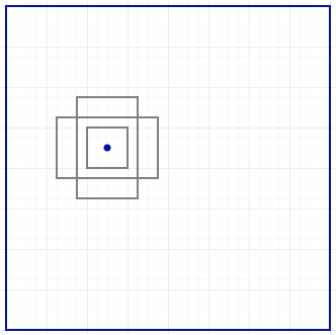
Volume di input



Posizione selezionata

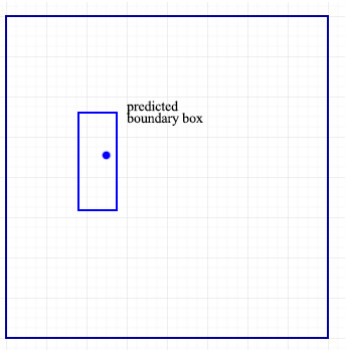


3 default box



$$\begin{bmatrix} \Delta x_1 \\ \Delta y_1 \\ w_1, \\ h_1, \\ conf_{1,1} \\ conf_{1,2} \\ \Delta x_2 \\ \Delta y_2 \\ w_2 \\ h_2 \\ conf_{2,1} \\ conf_{2,2} \\ \Delta x_3 \\ \Delta y_3 \\ w_3 \\ h_3 \\ conf_{3,1} \\ conf_{3,2} \end{bmatrix}$$

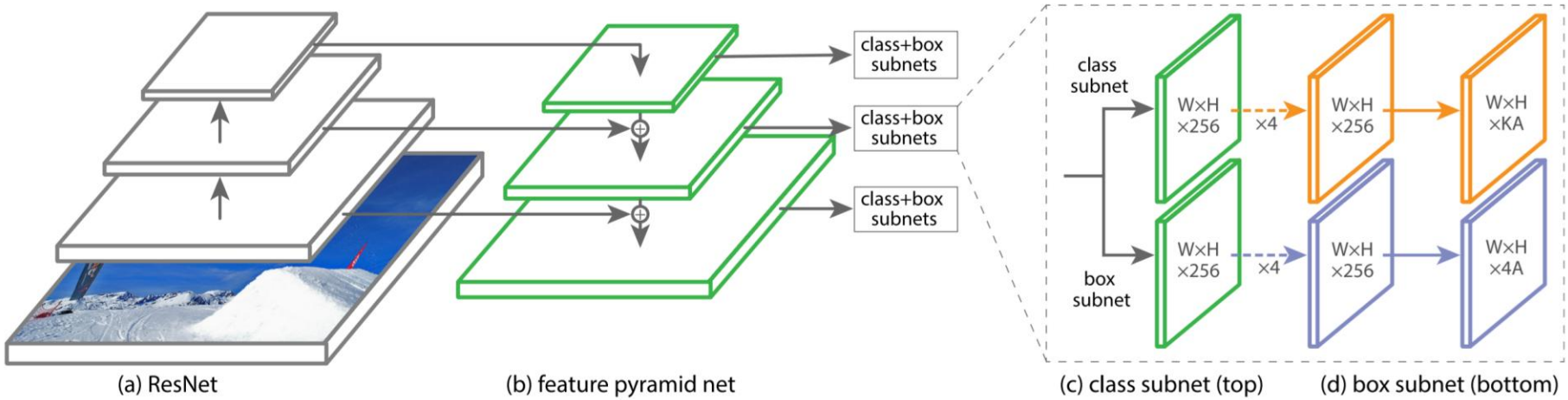
Default box 1
Default box 2
Default box 3



RetinaNet

Nel 2017, è stato proposto il modello single-stage RetinaNet costituito da tre componenti principali:

- 1 ResNet come backbone;
- 1 Feature Pyramid Network;
- 1 Detection head composta da due sottoreti.



T. Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollar, "Focal Loss for Dense Object Detection," *IEEE International Conference on Computer Vision (ICCV)*, October 2017.

RetinaNet (2)

- Utilizza una Feature Pyramid Network (FPN - che approfondiremo più avanti) per gestire oggetti di diverse scale combinando feature da più livelli.
- La Detection head è composta da due sottoreti: i) Classification subnet (per classificare gli oggetti) e ii) Regression subnet (per le coordinate delle bounding box).
- Utilizza anchor box di varie dimensioni e aspect ratio posizionate su ogni cella delle feature map per gestire oggetti di diverse scale e proporzioni senza dover generare region proposal.
- Utilizza Focal loss per bilanciare il peso di pochi oggetti (esempi difficili) rispetto allo sfondo (esempi facili).

RetinaNet (3)

Contributi

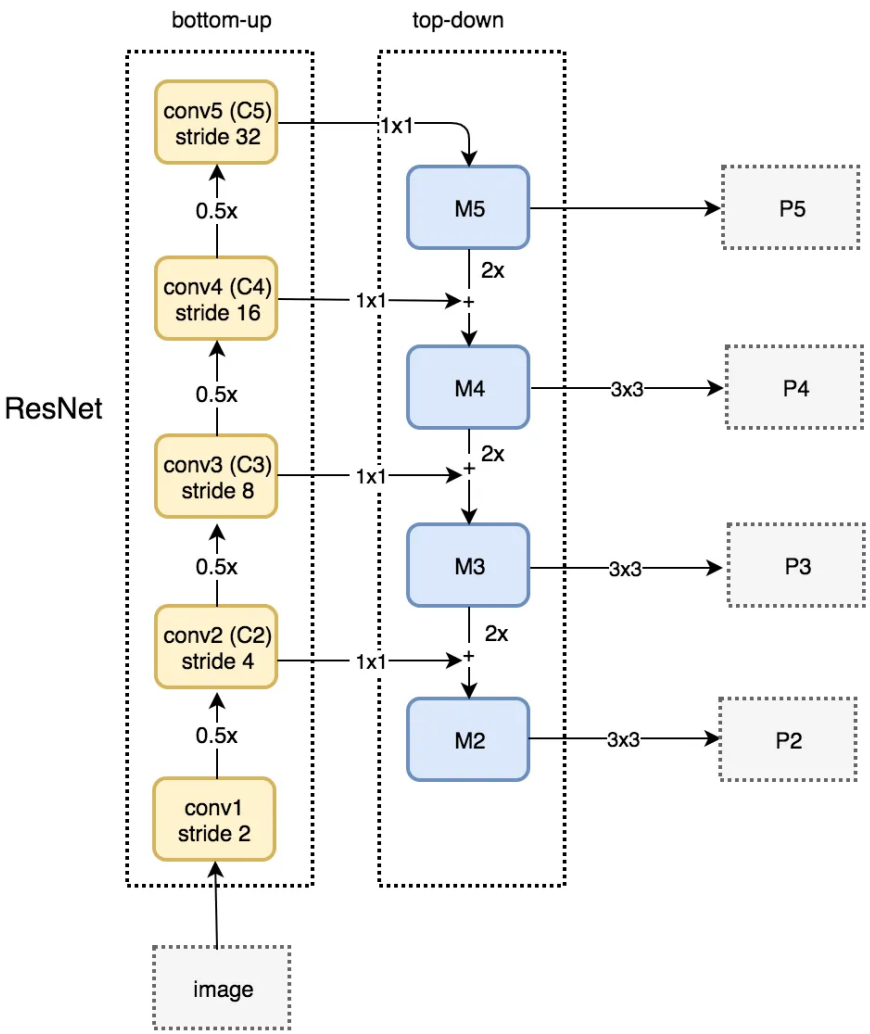
- Accuratezza confrontabile con Faster R-CNN.
- Feature Pyramid Network.
- Focal loss.

Svantaggi

- Più pesante di YOLO per applicazioni real-time.

Feature Pyramid Network

- Genera feature map a diverse risoluzioni (P2, P3, P4, P5) combinando informazioni provenienti da più livelli della rete.
- Migliora la localizzazione degli oggetti grazie a connessioni laterali tra i livelli ricostruiti e le corrispondenti mappe di feature della ResNet.
- Nel percorso top-down esegue un upsampling di fattore 2 del livello precedente utilizzando *nearest neighbor*.
- Riduce l'effetto di aliasing introdotto dalla fusione applicando una convoluzione con filtri 3×3.



Dataset di riferimento

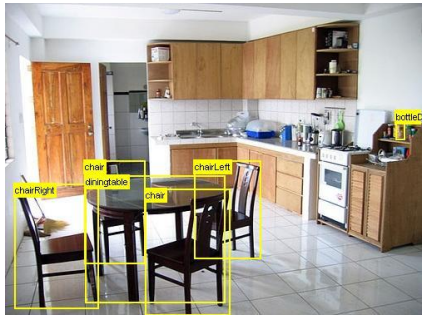
- COCO (Common Objects in Context)
  - Dimensione: ~330K immagini, 80 categorie.
  - Caratteristica: oggetti in contesti realistici.
  - Uso: benchmark principale per modelli allo stato dell'arte.
- PASCAL VOC
  - Dimensione: ~20K immagini, 20 categorie.
  - Caratteristica: dataset storico, semplice, ideale per introdurre concetti base.
  - Uso: training rapido e sperimentazioni didattiche.
- Open Images
  - Dimensione: >9M immagini, 600+ categorie.
  - Caratteristica: scene complesse con molti oggetti per immagine.
  - Uso: applicazioni su larga scala e modelli generalisti.

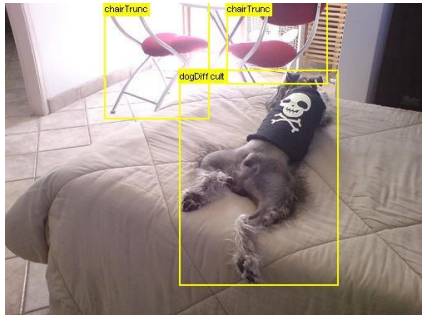
Modelli a confronto

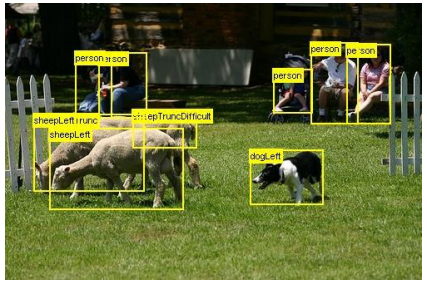
PASCAL Visual Object Classes Challenge 2012 è una competizione internazionale organizzata nel 2012 in cui i partecipanti competono per ottenere le migliori performance su diversi problemi di *computer vision* tra cui *object detection*.

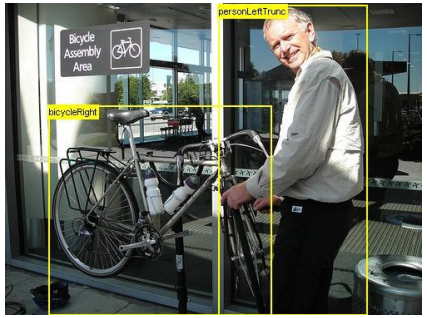
- 20 classi
- 11530 immagini di training+validation (con 27450 oggetti)
- 10991 immagini di test (con ? oggetti)

Model	mAP	Speed (ms/img)	Speed (FPS)
R-CNN	53.3	47000	0.02
Fast R-CNN	65.7	303	3.3
Faster R-CNN	73.2	200	5
YOLO	63.4	22	45
SSD	74.3	17	59
RetinaNet	77.1	198	5.1









Modelli disponibili

Oggi sono disponibili online numerosi modelli all'avanguardia, già addestrati su dataset di riferimento e dotati di pesi predefiniti pronti all'uso:

- YOLO – Ultralitics
- Faster R-CNN e Mask R-CNN – Detectron2
- DETR, RetinaNet, SSD e tantissimi altri modelli – MMDetection
- Vision Transformer – Hugging Face